

Memory Cache Simulator

Abhijit Kamble (2024CSB1401)

Tanish Kumar (2024CSB1161)

Course: CS204

Instructor: Prof. Neeraj Goel

Indian Institute of Technology (IIT) Ropar

1 Abstract

Modern processors rely heavily on cache memory to bridge the latency gap between the Central Processing Unit (CPU) and main memory. The efficiency of a cache system is primarily dictated by its architecture and the replacement policy used to manage its limited capacity. This project details the design and implementation of a trace-driven cache simulator. Using memory access traces generated dynamically via Intel Pin (Pintool), the simulator models cache behavior and evaluates the performance of four distinct replacement policies: Least Recently Used (LRU), First-In First-Out (FIFO), Random, and Belady's Optimal Policy.

2 Objectives

- **Dynamic Trace Generation:** Utilize Pintool to capture real-time memory access traces from executable programs.
- **Architecture Simulation:** Build a highly configurable simulator to model cache mapping, hits, misses, and evictions.
- **Policy Implementation:** Implement and comparatively analyze LRU, FIFO, Random, and Belady's Optimal replacement policies.
- **Performance Evaluation:** Measure cache efficiency using quantitative metrics, primarily focusing on the hit rate.

3 System Architecture

The simulation pipeline is divided into two decoupled stages: Data Collection and Simulation.

1. **Target Execution:** A C/C++ program is executed.
2. **Instrumentation:** Pintool hooks into the execution to log memory instructions.
3. **Trace Generation:** A raw trace file (`trace.out`) is produced.

4. **Simulation Engine:** The C++ simulator parses the trace, applies cache mapping logic, and executes replacement policies.
5. **Statistical Output:** Results are aggregated and output to the terminal and `statistics.out`.

4 Trace Generation and Parsing

Intel Pin was utilized to dynamically instrument the program and capture memory loads and stores.

Trace Format

The generated trace contains the instruction pointer, operation type, memory address, and access size.

[IP]	[OP]	[ADDRESS]	[SIZE]
0x714277025324	S	0x71427703ed98	8
0x714277025328	L	0x7ffc b14b7188	4

Data Utilization

For standard cache simulation, cache operations are managed at the block level rather than the byte level. Therefore, while modern SIMD instructions or varying data types may alter the access SIZE (e.g., 4, 8, or 16 bytes), all accesses falling within the same aligned block boundary map to the same cache line. Consequently, the simulator parses the raw trace but isolates the ADDRESS field to perform cache lookups, ignoring the operation type and size to streamline the simulation.

5 Cache Configuration and Mapping

The simulator is designed with a set-associative architecture defined by the following default parameters:

- **Cache Size (C):** 32768 bytes (32 KB)
- **Block Size (B):** 64 bytes
- **Associativity (A):** 4-way

Derived Architectural Parameters

- **Total Number of Blocks:** $Blocks = \frac{C}{B}$
- **Total Number of Sets:** $Sets = \frac{C}{B \times A}$
- **Offset Bits:** $\log_2(B)$
- **Index Bits:** $\log_2(Sets)$

Address Breakdown

Each 64-bit memory address parsed from the trace is logically partitioned into three segments using bitwise operations:

- **Offset:** Identifies the specific byte within the cache block.
- **Index:** Determines the specific set where the block can reside.
- **Tag:** A unique identifier to confirm if the cached block matches the requested memory address.

6 Simulation Logic

For every memory access in the trace, the simulation engine performs the following routine:

1. Parse the address to calculate the Tag and Set Index.
2. Iterate through the corresponding set's active blocks.
3. **Hit Condition:** If a valid block matches the calculated tag, a Hit is recorded. Metadata (such as LRU timestamps) is updated.
4. **Miss Condition:** If no tags match, a Miss is recorded.
 - If an invalid (empty) block exists in the set, the new block is inserted.
 - If the set is full, a Victim Block is selected using the designated replacement policy, an Eviction is recorded, and the new block replaces the victim.

7 Cache Replacement Policies

The system evaluates four distinct algorithms for victim selection during a cache miss in a full set.

7.1 Least Recently Used (LRU)

Mechanism: Evicts the block that has not been accessed for the longest period.

Implementation: A global timer increments on every access. Each block maintains a `last_used` timestamp, updated upon insertion and every subsequent hit. The block with the minimum timestamp in the set is selected for eviction.

Characteristics: Highly effective at exploiting temporal locality.

7.2 First-In First-Out (FIFO)

Mechanism: Evicts the oldest inserted block, regardless of how often it has been accessed since.

Implementation: Blocks maintain an `inserted_at` timestamp. The block with the minimum `inserted_at` value is evicted. Unlike LRU, this metadata is not updated upon a cache hit.

Characteristics: Simple to implement with low overhead, but can suffer from Belady's Anomaly in specific architectural configurations.

7.3 Random Replacement

Mechanism: Selects a victim block entirely at random.

Implementation: Utilizes a standard pseudo-random number generator modulo the associativity count.

Characteristics: Requires zero metadata storage (no timestamps). Serves as a fundamental baseline for evaluating the effectiveness of the intelligent policies.

7.4 Belady’s Optimal Policy

Mechanism: Evicts the block that will not be needed for the longest time in the future.

Implementation: Requires a preprocessing pass over the entire trace to build a hash map of queues (`unordered_map<ull, queue<int>>`), tracking the future occurrence indices of every address. During simulation, the algorithm peeks into these queues and evicts the block whose next use is furthest away. Blocks with no future uses are prioritized for immediate eviction.

Characteristics: Impossible to implement in real-time hardware due to its reliance on future knowledge. Used strictly as an analytical upper bound to measure how close practical algorithms (like LRU) get to theoretical perfection.

8 Performance Evaluation and Metrics

The primary metric for evaluation is the Hit Rate, calculated as:

$$\text{Hit Rate} = \left(\frac{\text{Hits}}{\text{Hits} + \text{Misses}} \right) \times 100$$

Simulation Results

The empirical results gathered from the trace simulation across the four evaluated policies are summarized below:

Policy	Hits	Misses	Evictions	Hit Rate (%)
LRU	192,318	7,682	6,658	96.159
FIFO	191,916	8,084	7,060	95.958
RANDOM	191,629	8,371	7,347	95.814
BELADY	188,844	11,156	10,132	94.422

Table 1: Cache Performance Statistics (32 KB, 4-Way Associative)

9 Observations and Analysis

- **Hit Rate Inflation:** Initial testing yielded exceptionally high hit rates ($> 95\%$). This is primarily due to the trace being heavily dominated by tight loops with strong temporal locality, and the 32 KB cache being large enough to hold the working set without forcing significant evictions.

- **Stack Accesses:** Filtering out stack reads and writes artificially inflates the hit rate by ignoring a large volume of valid, highly local memory operations.
- **Belady’s Anomaly / Implementation Artifact:** Theoretically, Belady’s Optimal policy yields the highest possible hit rate by acting as an absolute upper bound. However, the experimental data records Belady (94.422%) performing worse than LRU (96.159%). This anomaly points to an implementation constraint, boundary reset issue, or queue misalignment in the optimal policy’s predictive tracking logic across simulation runs, highlighting the practical complexities of modeling perfect future-knowledge algorithms.

10 Future Scope

- **Expanded Policy Support:** We will investigate for more replacement policies and implement them .
- **Data Visualization and Analytics:** we plan to generate visual comparisons—such as hit-rate versus cache capacity curves and miss-type distribution charts—for deeper statistical insights.
- **Benchmark Testing:** Evaluating the simulator against standardized, heavy-duty workload benchmarks to analyze replacement policy performance under real-world execution environments.

11 Conclusion

This project successfully models the internal mechanics of CPU cache memory using real-world execution traces. By isolating the simulation logic from the trace generation, the system provides a flexible environment for architectural analysis. The comparative data highlights the critical impact of replacement algorithms on memory latency, proving that practical predictive algorithms (like LRU) offer robust approximations for modern computing needs while tracking optimal benchmarks.